

RadeonShift AI

Accelerating the MI300X Hardware Migration via Generative AI

The Problem: CUDA Lock-in

Enterprise AI workloads are facing massive bottlenecks due to a reliance on legacy NVIDIA CUDA codebases.

- Manually migrating a 50,000-line CUDA codebase to ROCm takes **6+ months** of specialized engineering time.
- Portability tools exist (e.g. `hipify-perl`), but they lack the architectural awareness to optimize code for AMD Instinct architecture (like 64-wide wavefronts).
- Teams lack visibility into whether their migrated code will actually compile and run on AMD hardware without direct bare-metal access.

The Solution: RadeonShift AI

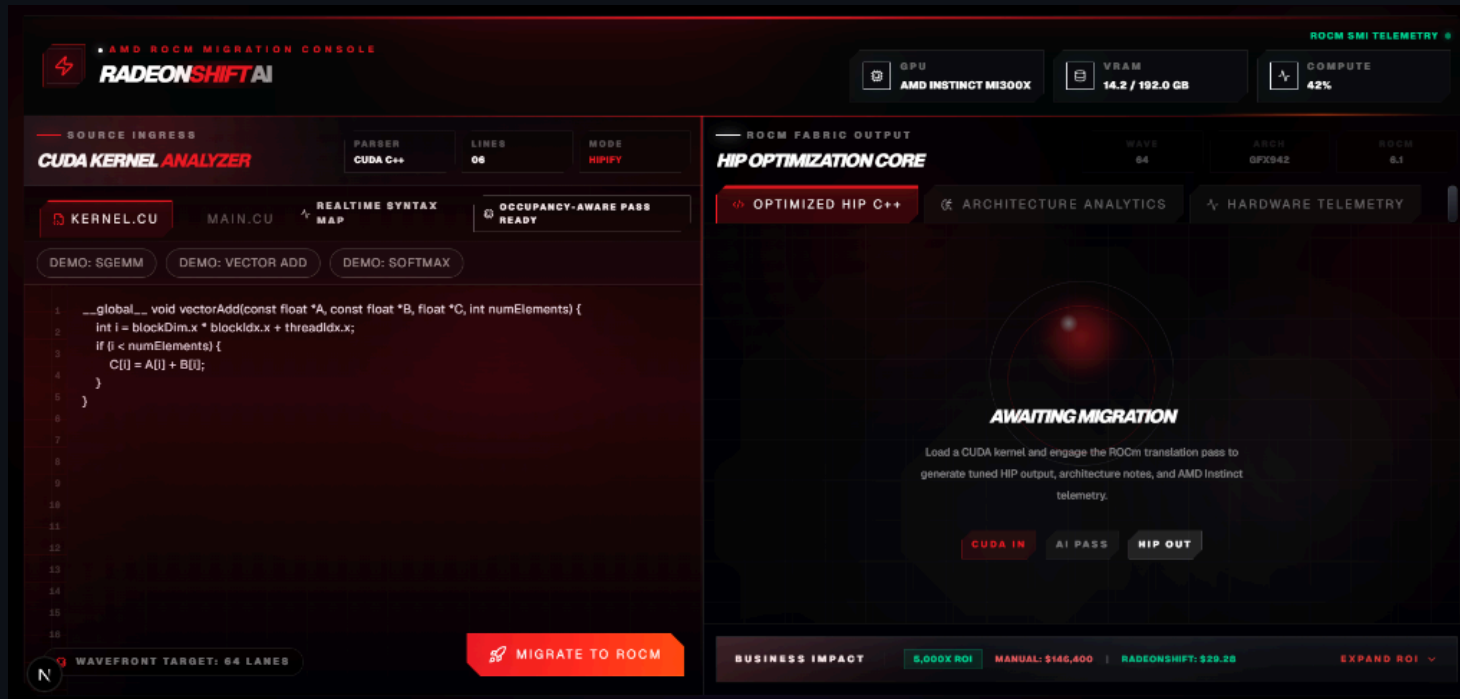
RadeonShift AI is an AI-assisted CUDA kernel migration assistant that translates CUDA to HIP/ROCm and audits architecture-specific migration risks.

1. **Syntax Translation:** Uses Fireworks-hosted translation model for AI Translation.
2. **MoA Audit:** Leverages DeepSeek-V4 to scan for PTX risks, hardcoded warp sizes, and AMD-specific optimizations.
3. **Optional Hardware Evidence:** When the ROCm backend is online, RadeonShift can compile-check generated HIP and show MI300X telemetry. Benchmark mode uses trusted internal kernels and clearly labels live, cached, or unavailable evidence.

Architecture

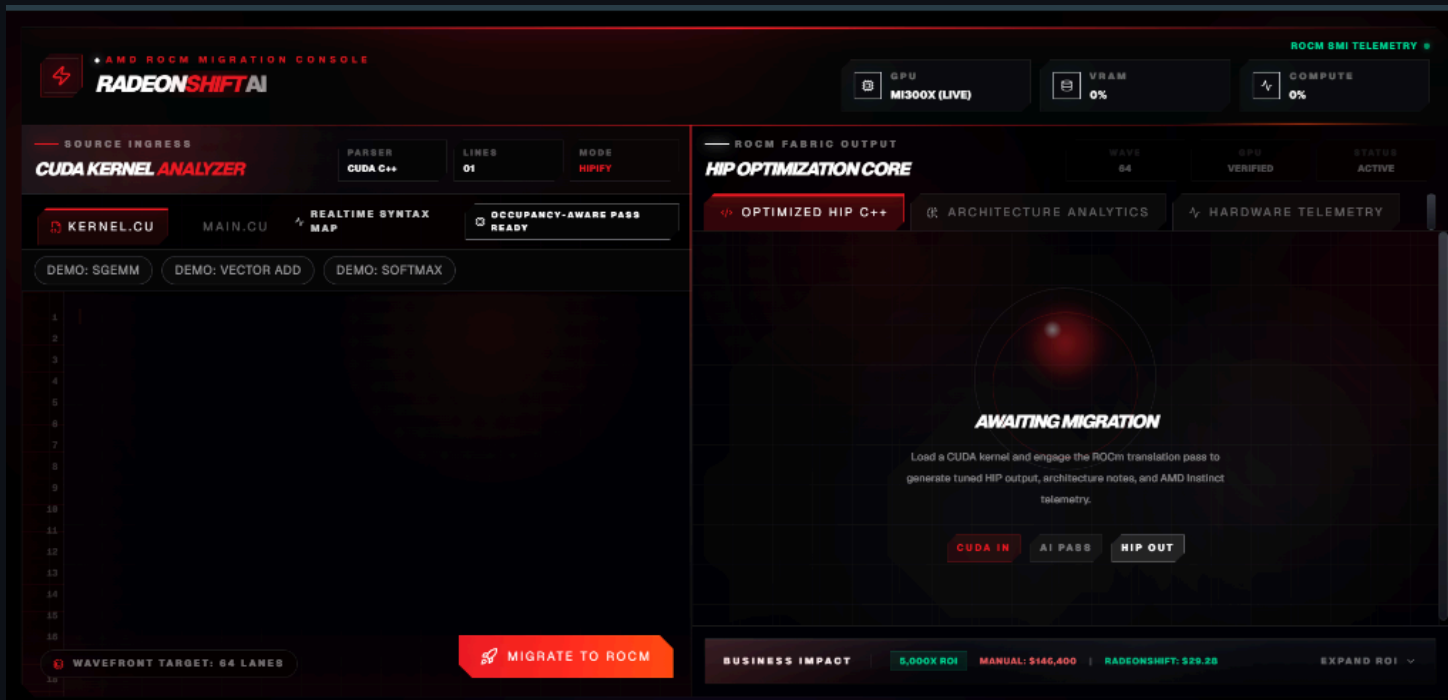
1. **Frontend (Next.js):** Manages user state and renders the dynamic dashboard.
2. **AI Translation Layer (Vercel / Next.js API Routes):** Orchestrates AI translation and Mixture-of-Agents audit via Fireworks AI, operating independently of hardware availability.
3. **Optional Hardware Engine (FastAPI via Pinggy):** A bare-metal ROCm layer that can compile-check generated HIP via `hipcc`, poll `rocm-smi` for telemetry, and run trusted benchmark kernels when connected.

Demo: The RadeonShift Dashboard



- **Source Code Editor:** Paste raw CUDA C++ kernels.
- **HIP Optimization Core:** View the translated target kernel.
- **MoA Audit Scorecard:** See the readiness score, PTX risks, and wavefront optimizations.

Step 1: The Awaiting Migration State



Before translation begins, the platform awaits the CUDA payload. The user inputs their native NVIDIA code into the **CUDA Kernel Analyzer**. The system prepares for the AI-assisted hardware pass.

Step 2: HIP Optimization Core

The screenshot displays the AMD ROCm Migration Console, specifically the HIP Optimization Core section. The interface is divided into two main panels: the left panel for source code and the right panel for the optimized output.

Left Panel (Source Code):

- Header:** AMD ROCm Migration Console, RADEONSHIFT AI. GPU: MI300X (LIVE), VRAM: 0%, COMPUTE: 0%.
- Navigation:** SOURCE INGRESS, CUDA KERNEL ANALYZER, PARSE, LINES: 278, MODE: HIPIFY.
- Kernel Info:** KERNEL.CU, MAIN.CU, REALTIME SYNTAX MAP, OCCUPANCY-AWARE PASS READY.
- Demos:** DEMO: SGEMM, DEMO: VECTOR ADD, DEMO: SOFTMAX.
- Code Snippet:**

```
222 float blockSum = 0.0f;
223 for (int i = 0; i < compactBlocks; i++) blockSum += h_blockResults[i];
224 printf("Cooperative reduce: %.2f (expected %.2f)\n", blockSum, totalSum);
225
226 printf("Volume density[0]: %.4f\n", d_densities[0]);
227 printf("Volume density[center]: %.4f\n", d_densities[volSize / 2]);
228
229 // ---- Cleanup ----
230 CHECK_CUDA(cudaDestroyTextureObject(texObj));
231 CHECK_CUDA(cudaFreeArray(cuArray));
232 CHECK_CUDA(cudaFree(d_edgeOutput));
233 CHECK_CUDA(cudaFree(d_input));
234 CHECK_CUDA(cudaFree(d_indices));
235 CHECK_CUDA(cudaFree(d_count));
236 CHECK_CUDA(cudaFree(d_blockResults));
237 CHECK_CUDA(cudaFree(d_volume));
238 CHECK_CUDA(cudaFree(d_densities));
239 free(h_img); free(h_data); free(h_edgeOutput); free(h_blockResults);
240
241 return 0;
```
- Buttons:** WAVEFRONT TARGET: 64 LANES, MIGRATE TO ROCM.

Right Panel (ROCm Fabric Output):

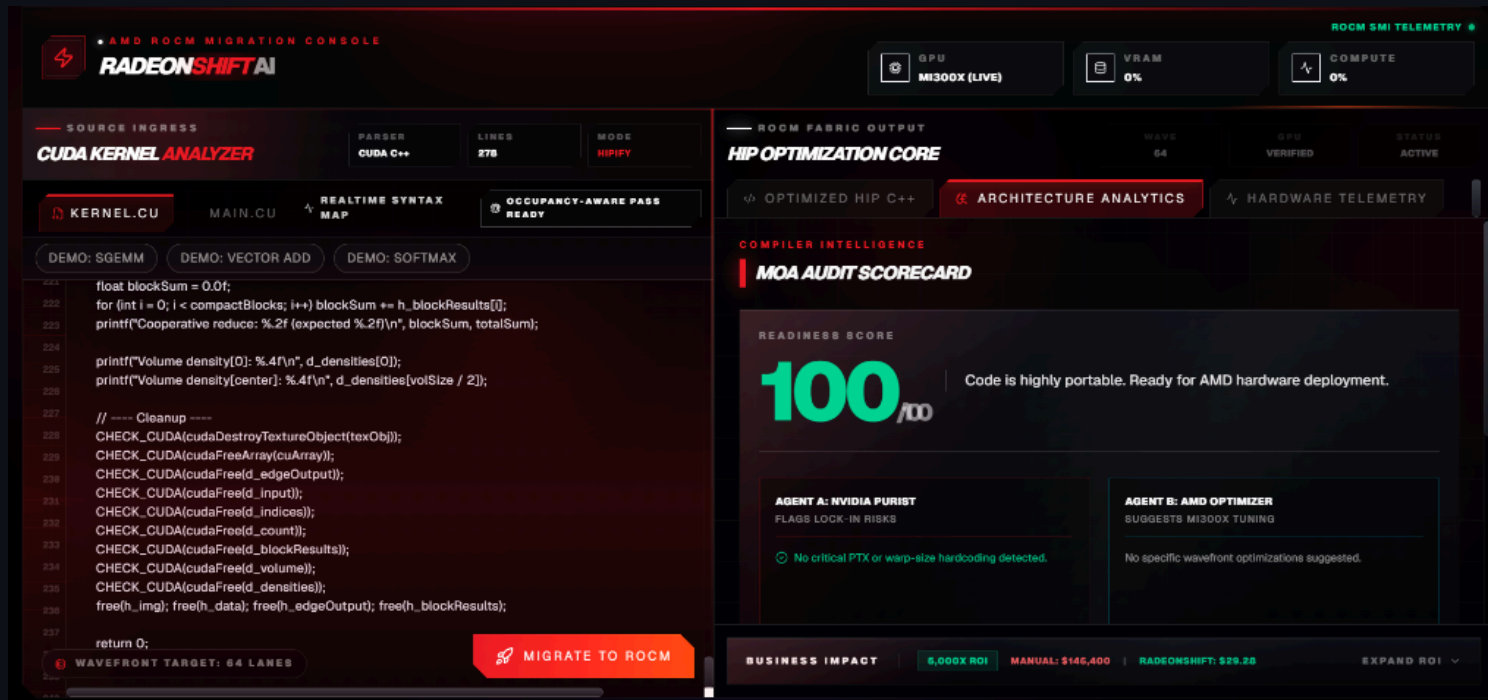
- Header:** ROCm Fabric Output, HIP OPTIMIZATION CORE, WAVE: 64, GPU: VERIFIED, STATUS: ACTIVE.
- Navigation:** OPTIMIZED HIP C++, ARCHITECTURE ANALYTICS, HARDWARE TELEMETRY.
- Generated Target:** TARGET_KERNEL.HIP.CPP, HIP TRANSLATION SUCCESSFUL.
- Code Snippet:**

```
#include "hip/hip_runtime.h"
#include <stdio.h>
#include <stdlib.h>
#include <hip/hip_runtime.h>
#include <hip/hip_cooperative_groups.h>
namespace cg = cooperative_groups;

#define CHECK_CUDA(call) do {
    hipError_t err = call;
    if (err != hipSuccess) {
        fprintf(stderr, "CUDA error: %s at %s:%d\n",
            hipGetErrorString(err), __FILE__, __LINE__);
        exit(1);
    }
}
```
- Business Impact:** 5,000X ROI, MANUAL: \$146,400, RADEONSHIFT: \$29.25, EXPAND ROI.

Upon engaging the ROCm translation pass, the core converts the syntax. The resulting C++ HIP code (`target_kernel.hip.cpp`) is immediately presented, demonstrating generated HIP translation.

Step 3: Architecture Analytics



The MoA Audit Scorecard evaluates the code and computes a readiness score from actual findings. Clean examples can score high, while risky kernels are lowered or capped when PTX, wavefront, WMMA, or async-copy risks appear.

Step 3a: Deterministic Redesign Guardrails

Advanced CUDA Kernel Detection

Not all CUDA code maps 1-to-1. RadeonShift's **Deterministic Rules Engine** catches unsupported architectures (e.g., CUDA WMMA, cooperative groups async-copy).

- **Correctness over completeness:** Safely enforces **MANUAL REDESIGN REQUIRED** if code relies on hardware-specific features.
- **Score Capping:** Drops readiness score to $< 50\%$ automatically, discouraging unsafe architecture conversions from being treated as ready.

Step 4: Live Hardware Telemetry

The screenshot displays the AMD ROCm Migration Console interface, which is divided into several sections. At the top, the 'AMD ROCm MIGRATION CONSOLE' header is visible, along with the 'RADEONSHIFT AI' logo. The top right corner shows 'ROCm SMI TELEMETRY' with status indicators for GPU (MI300X (LIVE)), VRAM (0%), and COMPUTE (0%).

The main interface is split into two primary panels. The left panel, titled 'SOURCE INGRESS', contains the 'CUDA KERNEL ANALYZER'. It shows a 'CUDA C++' file named 'KERNEL.CU' with 278 lines of code. The 'MODE' is set to 'HIPify'. Below the code editor, there are tabs for 'DEMO: SGEMM', 'DEMO: VECTOR ADD', and 'DEMO: SOFTMAX'. The code editor displays a CUDA kernel for a vector add operation, including a 'MIGRATE TO ROCm' button at the bottom.

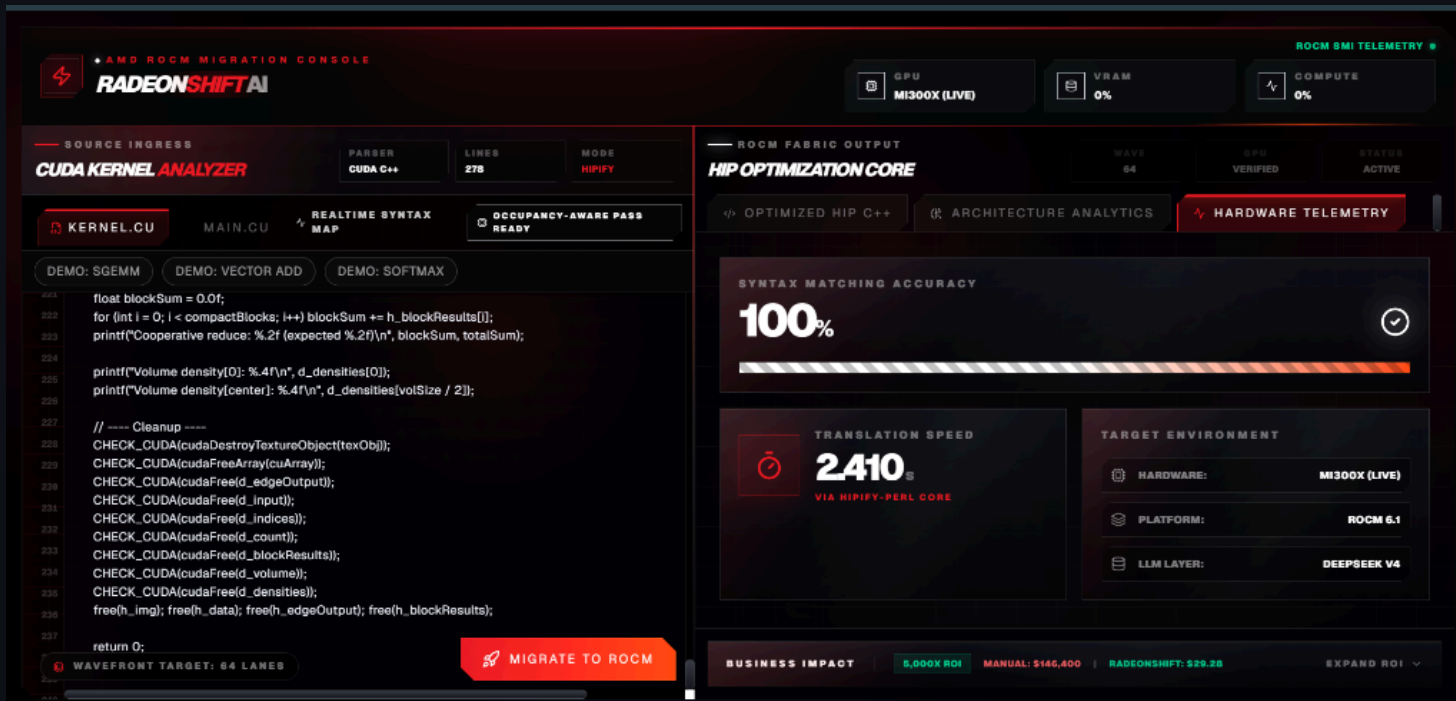
The right panel, titled 'ROCm FABRIC OUTPUT', contains the 'HIP OPTIMIZATION CORE'. It shows 'OPTIMIZED HIP C++' code and 'ARCHITECTURE ANALYTICS'. The 'HARDWARE TELEMETRY' section is active, displaying a 'LIVE HARDWARE TEST' for 'MI300X BENCHMARK MODE'. The test results show a 'BENCHMARK EXECUTION VERIFIED' status, with a 'COPY EVIDENCE' button. The benchmark results are summarized in a table:

ELAPSED TIME	THROUGHPUT	ELEMENTS	GPU
14.20 ms	5300.0 GB/s	16.8 M	AMD MI300X Bare-Metal

Below the benchmark results, a 'BUSINESS IMPACT' section shows a '6,000X ROI' compared to a 'MANUAL: \$146,400' cost, with 'RADEONSHIFT: \$29.28' as the optimized cost. An 'EXPAND ROI' button is also present.

When the optional backend is online, the platform connects to a remote bare-metal AMD MI300X environment and surfaces live ROCm telemetry plus compile-check evidence. When offline, hardware fields are explicitly labeled unavailable or cached.

Step 5: Bare-Metal Benchmark Execution



Finally, benchmark mode runs trusted HIP benchmark kernels on MI300X when hardware is online. RadeonShift does not automatically execute arbitrary uploaded kernels; it separates AI audit, compile-check evidence, and runtime benchmark provenance.

Business Value & ROI

Illustrative ROI Model

- **Manual Migration:** $244 \text{ Kernels} \times 4 \text{ hrs} = 976 \text{ Engineer-Hours}$ (~\$146,000, based on \$150/hr senior GPU engineer rate)
- **RadeonShift Migration:** $244 \text{ Kernels} \times \text{illustrative } \sim \$0.12 \text{ AI/compute cost} = \sim \29 first-pass triage estimate
- **Time Savings:** Initial audit and translation can shrink from weeks of first-pass review to minutes, with human validation still required for production.
- **TAM:** \$4.2B illustrative GPU migration and porting services market estimate.

The AMD Infrastructure Story

- **Hardware Target:** Optimized for AMD Instinct MI300X
- **Software Stack:** Built on ROCm 6.x APIs and `hipcc`
- **AI Acceleration:** MoA pipeline runs through the Fireworks-hosted inference API
- **Honest Fallbacks:** If the backend lacks ROCm hardware, the platform gracefully enters AI-Only mode, explicitly labeling any cached evidence without fabricating live metrics

Engineering Retrospective & Challenges

Bridging a Serverless Frontend with Bare-Metal Hardware

1. **Live Telemetry:** Defeated Next.js caching via `force-dynamic` to stream real-time MI300X metrics.
2. **Defensive APIs:** Built robust frontend parsing to gracefully handle sudden JSON schema changes.
3. **CORS Routing:** Bypassed strict mixed-content blocks by tunneling through Pinggy and Next.js `rewrites()`.
4. **Transparent Debugging:** Overhauled error handling to surface remote Python stack traces in the UI.
5. **Structured AI Prompts:** Constrained LLM outputs to raw HIP code or JSON audit findings, reducing UI parsing failures.

Closing & Team

RadeonShift AI is bridging the gap between legacy NVIDIA codebases and the future of AMD compute.

- GitHub Repository: [shashankh3/RadeonShift-AI](https://github.com/shashankh3/RadeonShift-AI)
- Live Demo: radeon-shift-ai.vercel.app

Thank You!

Shashank Hirwani

Unknown Hacker (shashankh366207)

